

Lecture 12:

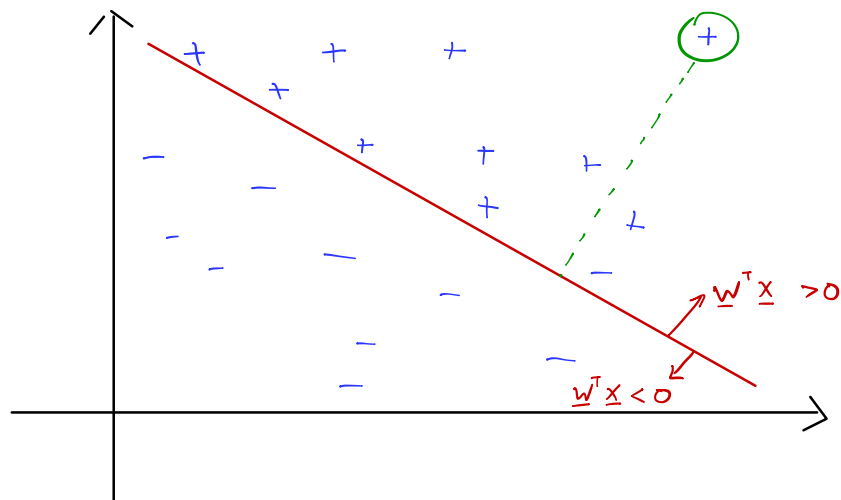
Support Vector Machines

Support Vector Machines

Assume labels $y_i \in \{+1, -1\}$

$$\text{Then } \|y - X\underline{w}\|_2^2 = \sum_{i=1}^n (y_i - \underbrace{x_i^T \underline{w}})^2 = \sum_{i=1}^n (1 - \underbrace{y_i x_i^T \underline{w}})^2$$

if this is $\neq 1$, then we incur loss even if point is correctly classified

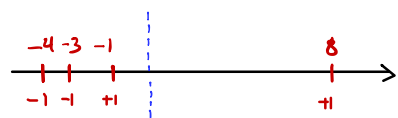


Ex. predict whether someone plays basketball based on height.

$y_i = +1 \rightarrow$ plays bball

$y_i = -1 \rightarrow$ doesn't play bball

4 training samples: x_i : 70', 71', 73', 82'
 y_i : -1, -1, +1, +1
 ← -4 ← -3 ← -1 ← 8 ← difference from mean 74'

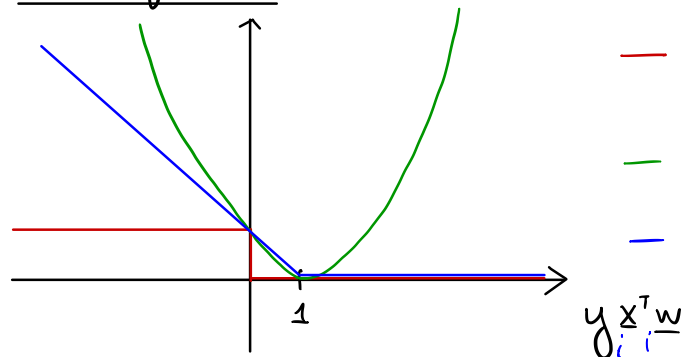


$$\hat{w}_{LS} = (\underline{X}^T \underline{X})^{-1} \underline{X}^T \underline{y} = 0.15 \Rightarrow \hat{y} = +1 \text{ if } (\text{height} - \text{mean}) > 0.15$$

LS decision boundary

misclassifies a point even though perfect classifier exists

Loss functions



some loss on some correctly classified points

no hinge loss, much quad loss

— 0/1 loss, $= \mathbb{I}\{y \neq \text{sign}(x^T w)\} = \mathbb{I}\{y x^T w < 0\}$ = "ideal loss"

— quadratic loss $= (1 - y x^T w)^2$

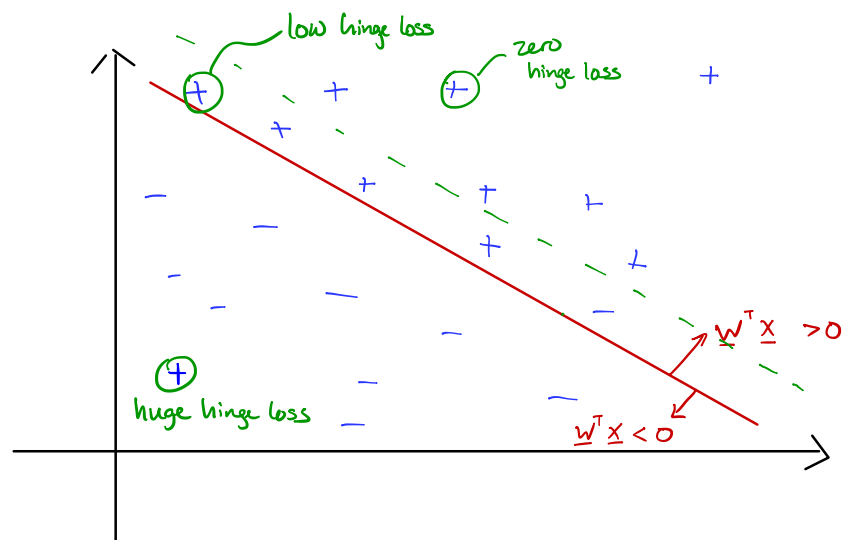
— hinge loss $= (1 - y x^T w)_+$

Hinge loss mimics the ideal loss but is convex and easy to minimize

if $y = +1$, $x^T w > 0$
 then we accurately predicted label
 if $y = -1$, $x^T w < 0$, acc. label

$$(a)_+ = \begin{cases} a & \text{if } a > 0 \\ 0 & \text{otherwise} \end{cases}$$

$$\mathbb{I}\{a\} = \begin{cases} 1 & \text{a true} \\ 0 & \text{a false} \end{cases}$$



$$l(\underline{w}) = \sum_{i=1}^n (1 - y_i x_i^T \underline{w})_+$$

$$\nabla_{\underline{w}} l = \sum_{i=1}^n \mathbb{I}\{y_i x_i^T \underline{w} < 1\} (-y_i x_i)$$

↑ technically called "subgradient" because $l(\underline{w})$ not differentiable

if $y x^T w > 0$, then correct label prediction
 $y x^T w < 0$, then wrong label prediction

If we minimize $\sum_{i=1}^n (1 - y_i x_i^T \underline{w})_+ + \lambda \|\underline{w}\|_2^2$

or the kernel version $\sum_{i=1}^n (1 - y_i \phi(x_i)^T \underline{w})_+ + \lambda \|\underline{w}\|_2^2$

this is called a support vector machine

$$\text{Let } \hat{\underline{w}}_{svm} = \underset{\underline{w}}{\operatorname{argmin}} \sum_{i=1}^n (1 - y_i \underline{x}_i^T \underline{w})_+ + \lambda \|\underline{w}\|_2^2$$

$$\text{or } \hat{\underline{w}}_{svm} = \underset{\underline{w}}{\operatorname{argmin}} \sum_{i=1}^n (1 - y_i \phi(\underline{x}_i)^T \underline{w})_+ + \lambda \|\underline{w}\|_2^2$$

Just like with kernel ridge regression, it is possible to show $\hat{\underline{w}}_{svm} = X^T \underline{\alpha}$ for some $\underline{\alpha}$ (different from least squares $\underline{\alpha}$)

$$\text{or } \hat{\underline{w}}_{svm} = \Phi^T \underline{\alpha}$$

To see this:

$$\text{Imagine } \hat{\underline{w}} = X^T \underline{\alpha} + \underline{w}^\perp = \sum_{j=1}^n \alpha_j \underline{x}_j + \underline{w}^\perp \quad (\underline{w}^\perp \text{ some vector orthogonal to the } \underline{x}_i\text{'s})$$

$$(a)_+ = \begin{cases} a & \text{if } a \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

$$\text{then } \min_{\underline{\alpha}, \underline{w}^\perp} \sum_{i=1}^n (1 - y_i \underline{x}_i^T (\sum_{j=1}^n \alpha_j \underline{x}_j + \underline{w}^\perp))_+ + \lambda \left\| \sum_{j=1}^n \alpha_j \underline{x}_j + \underline{w}^\perp \right\|_2^2$$

$$= \min_{\underline{\alpha}, \underline{w}^\perp} \sum_i (1 - y_i [\sum_{j=1}^n \alpha_j \langle \underline{x}_i, \underline{x}_j \rangle + \underbrace{\underline{x}_i^T \underline{w}^\perp}_{\text{always zero!}}])_+ + \lambda \left[\left\| \sum_{j=1}^n \alpha_j \underline{x}_j \right\|_2^2 + \|\underline{w}^\perp\|_2^2 \right]$$

$$= \min_{\underline{\alpha}, \underline{w}^\perp} \sum_i (1 - y_i [\sum_{j=1}^n \alpha_j \langle \underline{x}_i, \underline{x}_j \rangle])_+ + \lambda \left[\left\| \sum_{j=1}^n \alpha_j \underline{x}_j \right\|_2^2 + \|\underline{w}^\perp\|_2^2 \right]$$

$\rightarrow$ this must be 0 at optimum!

$$\hat{\underline{\alpha}} = \underset{\underline{\alpha}}{\operatorname{argmin}} \sum_i (1 - y_i [\sum_{j=1}^n \alpha_j \langle \underline{x}_i, \underline{x}_j \rangle])_+ + \lambda \underbrace{\left\| \sum_{j=1}^n \alpha_j \underline{x}_j \right\|_2^2}_{= \sum_i \sum_j \alpha_i \alpha_j \langle \underline{x}_i, \underline{x}_j \rangle}$$

Alternatively, we can apply the "kernel trick" and replace inner products with kernels:

$$\hat{\underline{\alpha}} = \underset{\underline{\alpha}}{\operatorname{argmin}} \sum_{i=1}^n (1 - y_i \sum_j \alpha_j K(\underline{x}_i, \underline{x}_j))_+ + \lambda \sum_i \sum_j \alpha_i \alpha_j K(\underline{x}_i, \underline{x}_j)$$

There is no closed form solution to this optimization problem.

$\Rightarrow$ need to use gradient descent or other numerical optimization methods.

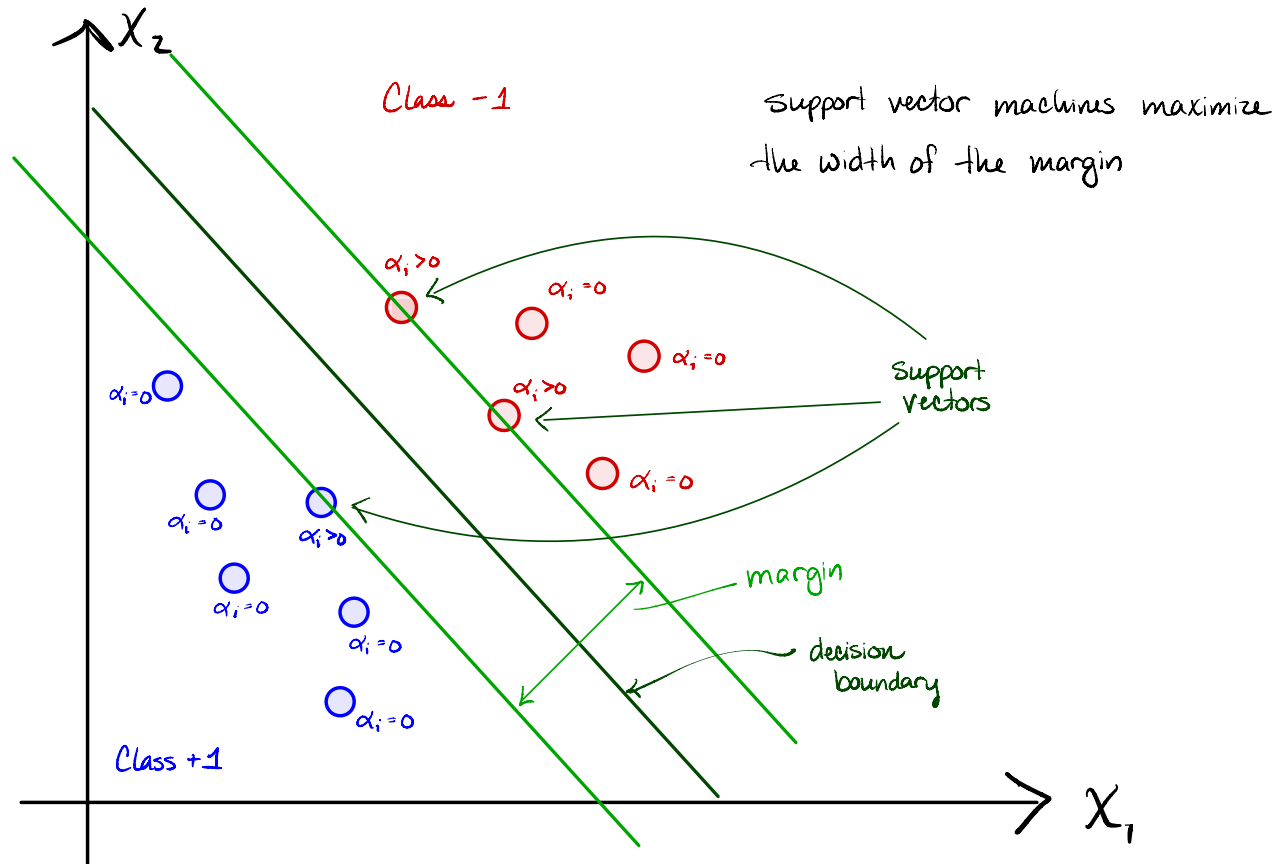
Why is this called a "support vector machine"?

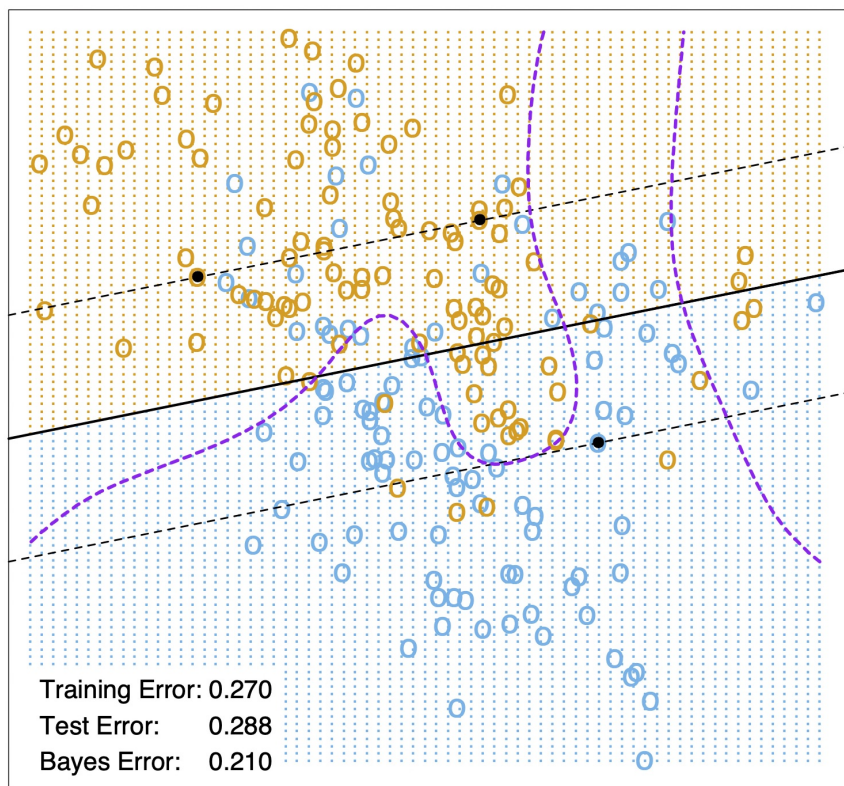
Typically, $\underline{\hat{\alpha}}$ is sparse — most $\hat{\alpha}_j = 0$

Recall $\hat{\underline{w}} = \sum_{j=1}^n \hat{\alpha}_j \phi(\underline{x}_j)$

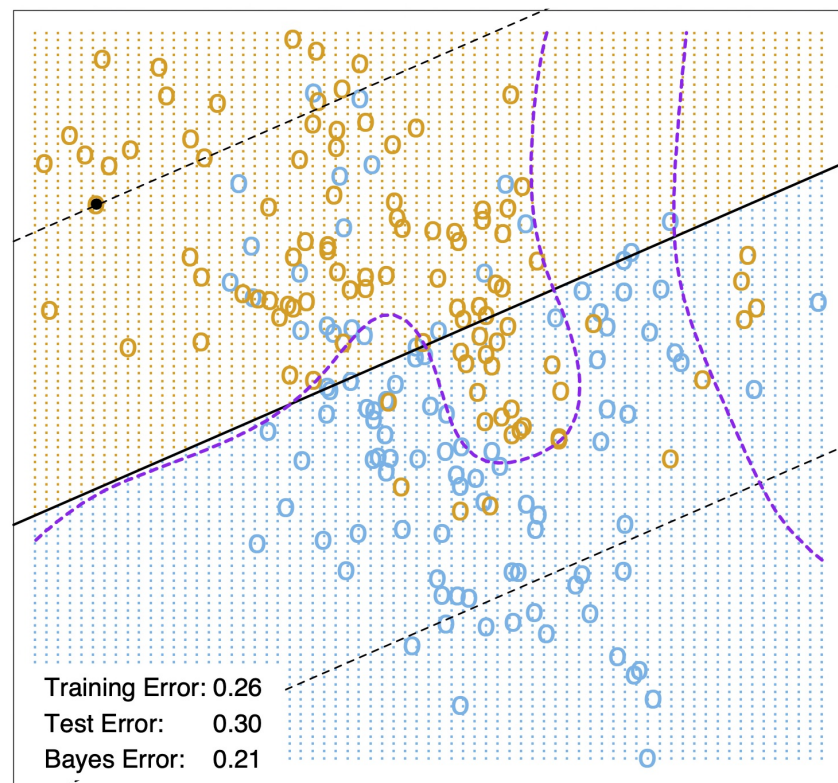
$\Rightarrow \hat{w}$ is a linear combination of only a few training samples (in feature space)

those $\underline{x}_i$'s are called support vectors.





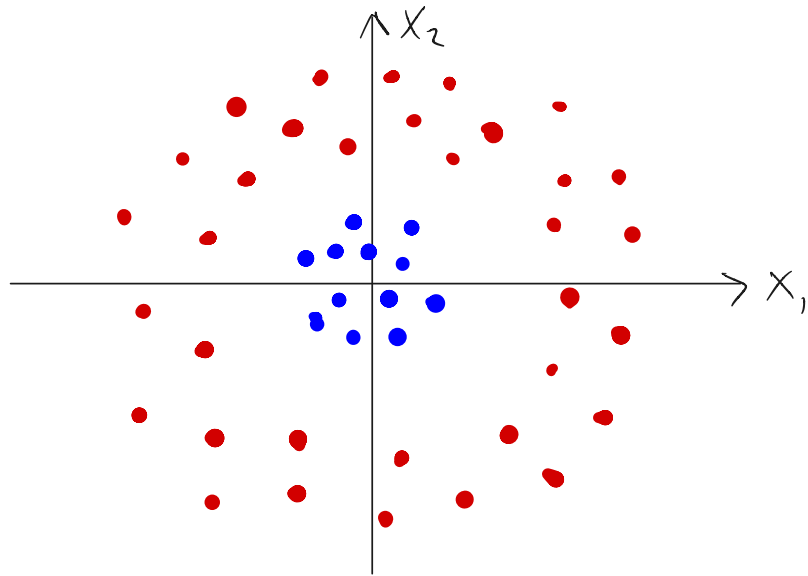
$$\lambda = 10^{-4}$$



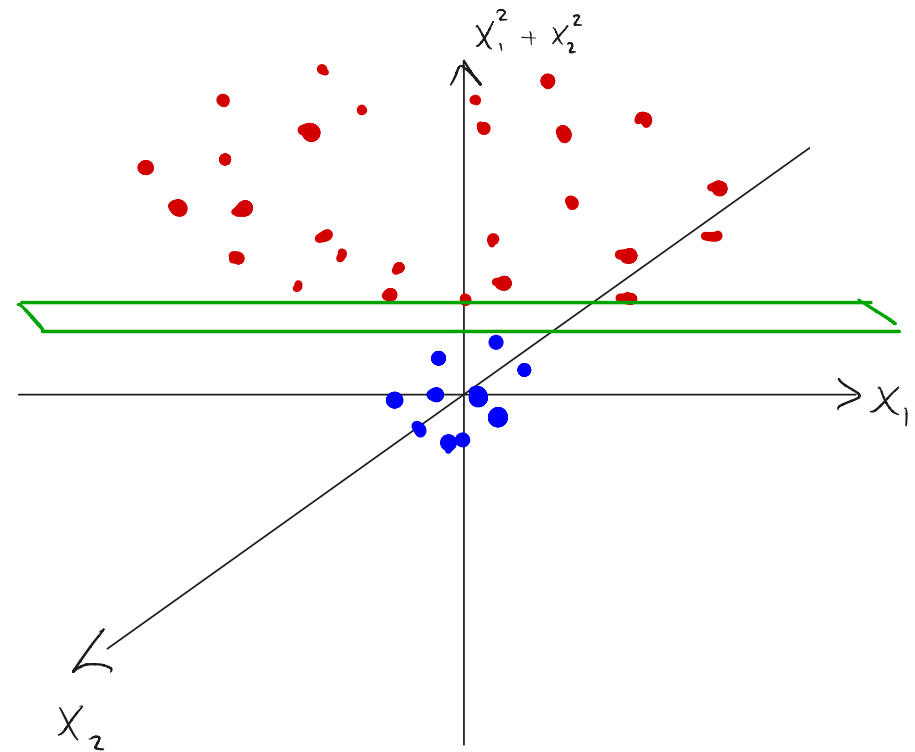
$$\lambda = 100$$

- ideal ("Bayes") decision boundary — gives smallest test errors on average
- points $\underline{x}$ where $\hat{y} = \underline{x}^T \hat{\underline{w}}_{svm} = \pm 1$
- SVM decision boundary

How do kernels help?

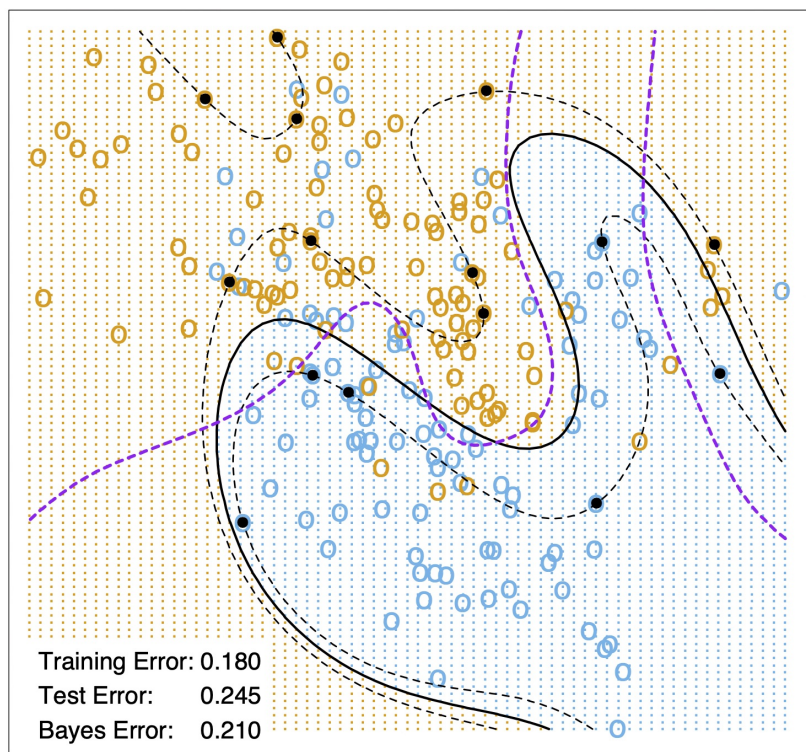


no good linear classifier exists

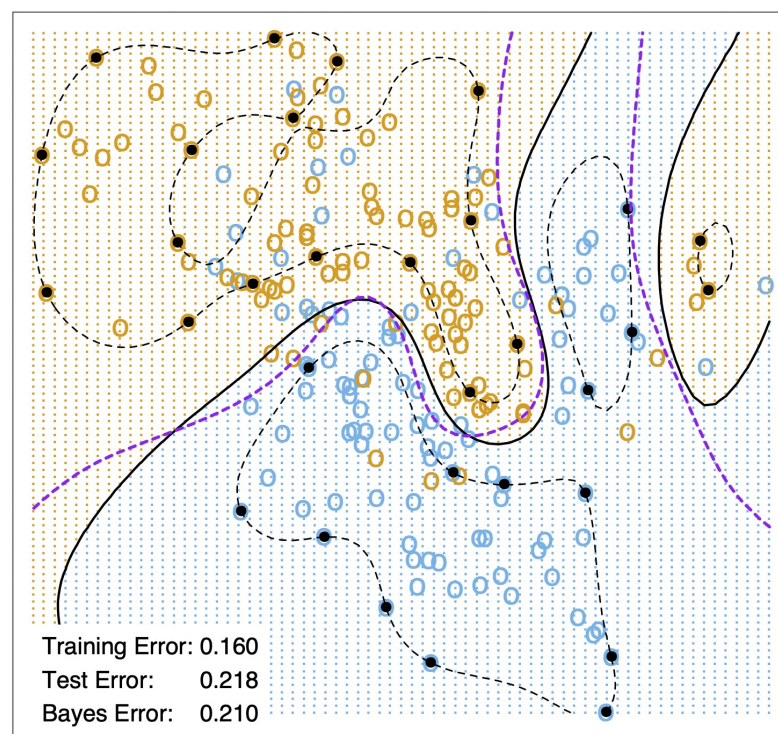


in high-dimensional feature space, a good linear separating hyperplane exists.

SVM - Degree-4 Polynomial in Feature Space



SVM - Radial Kernel in Feature Space



- ideal ("Bayes") decision boundary — gives smallest test errors on average
- points x where $\hat{y} = x^T \hat{w}_{svm} = \pm 1$
- SVM decision boundary